



Introdução a Teste de Software

Herysson R. Figueiredo
herysson.figueiredo@ufn.edu.br



Objetivos da aula

Compreender a relação entre qualidade, V&V e teste de software.

Definir teste de software e reconhecer sua finalidade.

Distinguir os níveis de teste: unidade, integração, validação e sistema.

Definir caso de teste, cenário/suíte de teste e registro de defeito.

Diferenciar engano, defeito, erro e falha.



Para começar: o que é software de qualidade?

“Você conhece um software de qualidade?”

No que você se baseou para julgar a qualidade?

O que é qualidade de software?

Como a atividade de teste contribui para atingir e avaliar essa qualidade?



Qualidade de *software*

“Qualidade de software pode ser definida como: uma gestão de qualidade efetiva aplicada de modo a criar um produto útil que forneça valor mensurável para aqueles que o produzem e para aqueles que o utilizam.”

ISO/IEC 25010:2023

Adequação funcional	Eficiência de desempenho	Compatibilidade	Capacidade de interação	Confiabilidade	Segurança	Manutenibilidade	Flexibilidade	Segurança Operacional
Compleitude Funcional Correção funcional Adequação funcional	Comportamento Temporal Utilização de recursos Capacidade	Coexistência Interoperabilidade	Reconhecibilidade de adequação Aprendizagem Operabilidade Proteção contra Erros do Usuário Engajamento do usuário Inclusividade Assistência ao usuário Auto-descritividade	Ausência de Falhas Disponibilidade Tolerância a falhas Recuperabilidade	Confidencialidade Integridade Não-repúdio Prestação de contas Autenticidade Resistência	Modularidade Reutilização Analisabilidade Modificability Testabilidade	Adaptabilidade Escalabilidade Instalação Substituibilidade	Restrição operacional Identificação de risco Falha Segura Alerta de Perigo Integração segura

Fonte: Adaptado de [ISO/IEC 25010](#)

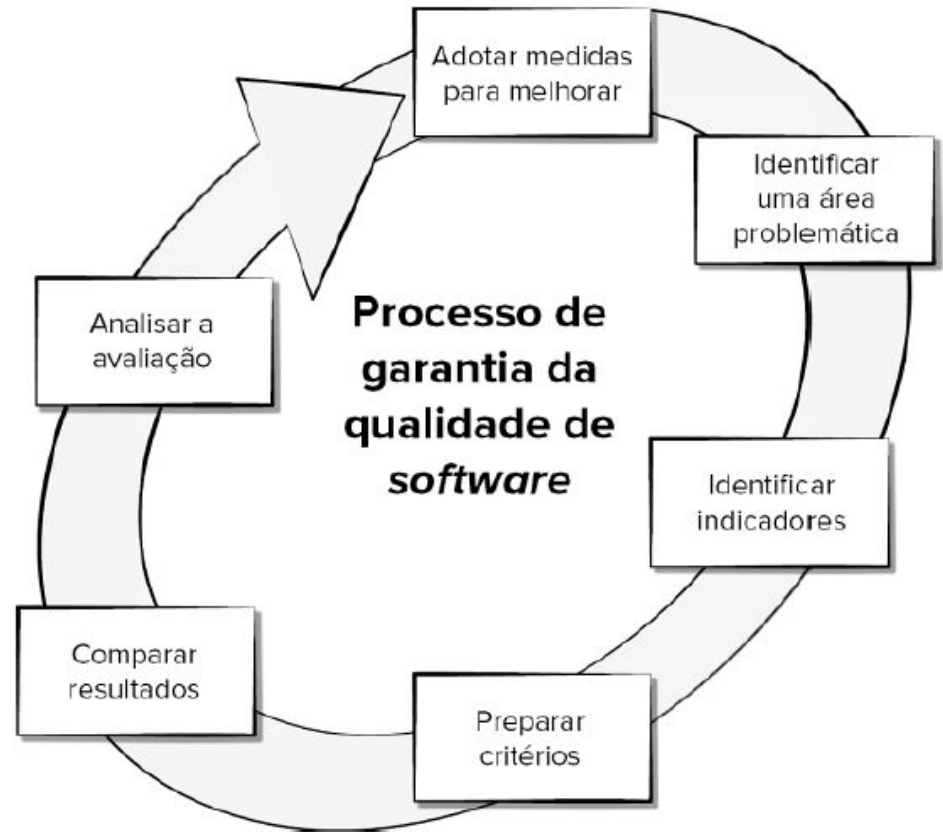


SQA (Software Quality Assurance)

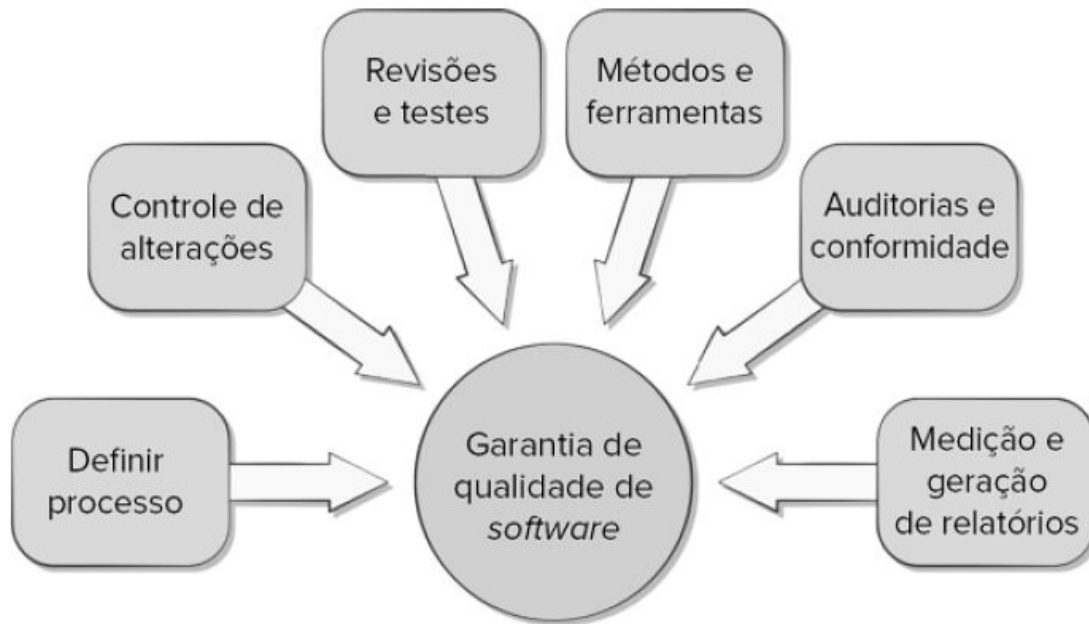
Sempre que falamos de Qualidade de *Software* devemos lembrar nas da **garantia de qualidade de software – SQA (Software Quality Assurance)**. Que pode ser definida como um padrão de ações planejado e sistematizado necessário para garantir alta qualidade no software

Pressman (PRESSMAN, 2021)

Processo da SQA:



SQA (Software Quality Assurance)





Elementos da SQA (*Software Quality Assurance*)

Coleta e análise de erros/defeitos: A SQA coleta e analisa dados sobre erros e defeitos para entender melhor como eles são introduzidos e como podem ser eliminados.

Gerenciamento de mudanças: A SQA garante que práticas adequadas de gerenciamento de mudanças sejam instituídas para evitar a confusão que leva a uma qualidade inadequada.

Educação: A SQA assume a liderança na melhoria do processo de software e patrocina programas educacionais para engenheiros e gerentes.



Elementos da SQA (*Software Quality Assurance*)

Gerência dos fornecedores: A SQA é responsável por garantir que o software adquirido de fornecedores externos atenda aos padrões de qualidade.

Gestão de segurança: A SQA garante o emprego de processos e tecnologias apropriados para a segurança do software.

Proteção: A SQA pode ser responsável por avaliar o impacto de falhas de software e iniciar etapas para a redução de riscos.

Gestão de riscos: O grupo de SQA garante que as atividades de gestão de riscos sejam conduzidas apropriadamente e que planos de contingência tenham sido estabelecidos.



Elementos da SQA (*Software Quality Assurance*)

Padrões: A SQA garante que os padrões adotados sejam seguidos e que os produtos resultantes estejam em conformidade com eles.

Revisões e auditorias: As revisões técnicas são realizadas para revelar erros, enquanto as auditorias são conduzidas pelo pessoal de SQA para assegurar que as diretrizes de qualidade estão sendo seguidas.

Teste: A SQA usa o teste como uma das atividades mais importantes para avaliar a qualidade.



Teste faz parte da garantia da qualidade

O teste é uma atividade importante de V&V/SQA, mas não “cria” qualidade depois que o produto está pronto.

- A qualidade deve ser incorporada ao longo do processo de software.
- O teste fornece evidências sobre defeitos, conformidade e comportamento do produto.
- Revisões técnicas, auditorias e outras atividades de SQA complementam os testes.



Verificação e Validação (V&V)

VERIFICAÇÃO

**Estamos construindo o produto
corretamente?**

VALIDAÇÃO

Estamos construindo o produto certo?

V&V abrange diferentes atividades de garantia da qualidade, entre elas revisões, inspeções, testes e outras formas de avaliação.



Atividades estáticas e dinâmicas

ESTÁTICAS

Realizadas sem depender de um artefato executável.

Revisões técnicas e inspeções são exemplos.

DINÂMICAS

Dependem de um artefato executável.

Teste de software é uma atividade dinâmica.

Planejar testes pode começar antes do código; executar o teste exige um artefato executável.



O que é teste de software?

Executar ou simular um artefato sob condições conhecidas, observar os resultados e compará-los com o comportamento esperado.

- Entradas e pré-condições precisam ser conhecidas ou controladas.
- O resultado obtido é confrontado com o resultado esperado.
- O teste produz evidências sobre conformidade com requisitos e comportamento do sistema.



Teste de *software*

“Teste é a simulação de um artefato executável com entradas e pré-condições conhecidas e a comparação entre a saída atual e a saída esperada para determinar se o artefato está em conformidade com os requisitos” (FIRESMITH, 2013).

“Teste é o processo de executar um sistema ou componente sob condições específicas, observando e registrando os resultados, avaliando alguns aspectos do sistema ou componente” (ISO/IEC/IEEE 24765, 2010).

“Teste é o processo de executar um programa ou sistema com a intenção de encontrar erros” (MYERS, 2004).



Mentalidade de teste: procurar defeitos

Testar não significa provar que o software “não tem defeitos”.

O que podemos afirmar é mais limitado: para as entradas, condições e caminhos exercitados, observamos determinado comportamento.

Por isso, o objetivo do teste é encontrar defeitos e produzir informação útil sobre a qualidade do produto.



Características de uma estratégia de teste

- Revisões técnicas eficazes antecedem os testes e ajudam a remover erros antes da execução.
- O teste começa no nível de componente e progride em direção à integração do sistema.
- Diferentes técnicas de teste são adequadas a diferentes momentos do desenvolvimento.
- Testes são realizados por desenvolvedores e, em projetos maiores, por grupos independentes.
- Teste e depuração são atividades distintas, embora relacionadas.



Quem testa o software?

Desenvolvedor

- Responsável pelo teste das unidades/componentes que desenvolve.
- Em muitos contextos também participa do teste de integração.

Grupo independente de teste

- Atua após a arquitetura estar suficientemente definida.
- Ajuda a reduzir conflitos de interesse e trabalha em conjunto com a equipe de desenvolvimento.

Qualidade e teste são responsabilidades compartilhadas; independência não significa isolamento.



Três ideias equivocadas sobre teste

MITO 1

“O desenvolvedor não deve testar seu próprio software.”

MITO 2

“Depois de pronto, o software deve ser simplesmente entregue ao time de teste.”

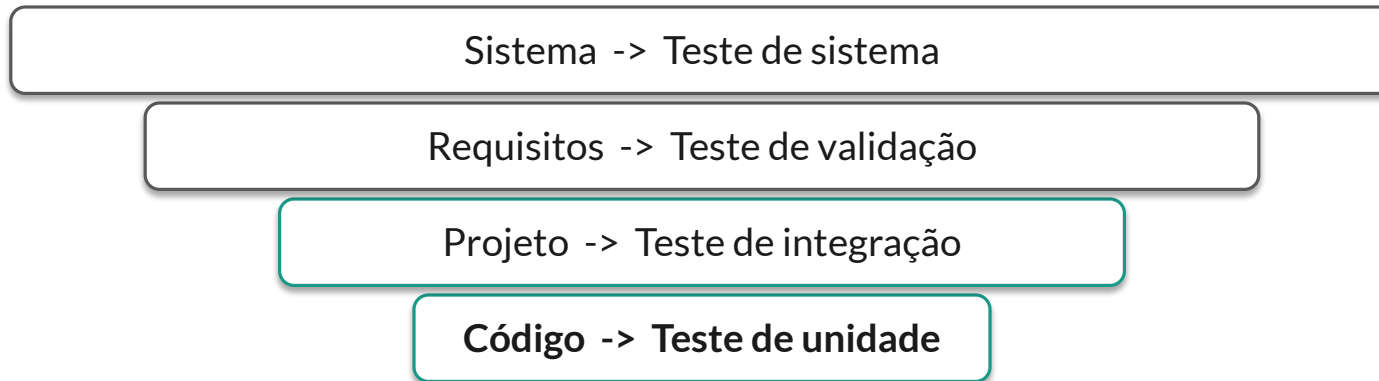
MITO 3

“Testadores só entram no projeto quando os testes começam.”

Pressman e Maxim tratam as três afirmações como incorretas: teste precisa ser planejado, integrado ao processo e compartilhado entre papéis.



Teste acompanha o desenvolvimento



A estratégia parte do “pequeno” (componentes) e amplia o escopo até o sistema completo.



Níveis de teste - visão geral

Nível	Foco principal	Base típica
Unidade	Menor unidade de software	Código / componente
Integração	Interfaces e comunicação entre unidades	Arquitetura / projeto
Validação	Requisitos e necessidades do usuário	Requisitos / casos de uso
Sistema	Software no sistema completo e ambiente	Sistema integrado



Teste de unidade

- Dedicar esforço à menor unidade do sistema: componente, módulo, classe ou método.
- Examina lógica interna, fluxo de dados, condições-limite e tratamento de erros.
- O escopo reduzido facilita a localização e a depuração de defeitos.
- *Drivers* e *stubs* podem ser necessários para simular unidades adjacentes.

Stub: Simula um módulo inferior (dependência) que ainda não existe, retornando respostas fixas ou pré-programadas para o código em teste.

Driver: Simula um módulo superior (chamador) que ainda não existe, acionando a função que você quer testar e exibindo seus resultados.



Teste de integração

Unidades testadas isoladamente não garantem que funcionarão corretamente quando combinadas.

- Constrói a arquitetura de forma sistemática e busca defeitos associados às interfaces entre unidades.
- Pode ser realizado de forma incremental, adicionando módulos gradualmente.
- A integração incremental facilita o isolamento de defeitos em relação à abordagem “big-bang”.



Teste de validação

Pergunta central: o sistema desenvolvido satisfaz os requisitos e as necessidades do usuário?

- Os casos de teste são guiados principalmente pelos requisitos funcionais e cenários de uso.
- Avalia comportamento, desempenho, interface, usabilidade e documentação em relação ao esperado.
- O foco está nas ações do usuário e nas saídas observáveis do sistema.



Teste de sistema

O software é exercitado como parte de um sistema maior: hardware, dados, pessoas, dispositivos e outros serviços.

- Verifica se os elementos do sistema funcionam em conjunto de acordo com os objetivos globais.
- O ambiente e suas configurações passam a fazer parte do problema de teste.
- Entre os testes de sistema, Martins apresenta segurança, recuperação, desempenho e stress.



Do componente ao sistema



O escopo cresce progressivamente: primeiro uma unidade, depois conjuntos de unidades, os requisitos do produto e, por fim, o sistema completo.

Em processos iterativos e ágeis, esse movimento pode se repetir a cada incremento.



Planejamento dos testes

- Planejar o teste antes de depender apenas da execução do sistema completo.
- Definir requisitos do produto de forma quantificável antes do teste.
- Definir objetivos de teste explicitamente e conhecer os usuários/categorias de usuário.
- Projetar software testável e usar revisões técnicas como filtro antes da execução.
- Manter um plano de teste que evolua com o projeto.



Manutenção de registros de teste

Registrar testes não precisa ser oneroso - mas precisa ser suficiente para reproduzir, acompanhar e aprender.

- Identificação do caso de teste e conteúdo relevante.
- Resultado: aprovação ou reprovação.
- Data da execução e comentários sobre falhas/defeitos.
- Registros podem apoiar depuração, acompanhamento e manutenção futura.



Teste exaustivo: por que não?

Testar todas as combinações possíveis de entradas, estados e caminhos é impraticável para sistemas não triviais.

- O número de combinações cresce rapidamente.
- O custo e o tempo de execução tornam a exaustão inviável.
- Por isso, critérios e técnicas de teste ajudam a selecionar casos com maior potencial de revelar defeitos.



Teste exaustivo: por que não?

```
int exemplo_simples(int j) {  
    j = j - 1;  
    j = j / 30000;  
    return j;  
}
```



A escolha dos dados de teste importa

Um conjunto de dados pode revelar um defeito; outro conjunto pode não exercitar o comportamento defeituoso.

No exemplo discutido anteriormente, um domínio com 65.536 valores possíveis ilustra como poucos valores específicos podem expor o problema.

DOMÍNIO

**65.536 valores
possíveis**

VALORES CRÍTICOS

**poucos valores
podem revelar o
defeito**



De onde surgem os casos de teste?

Requisitos, histórias de usuário, casos de uso e critérios de aceitação orientam a definição dos testes.

- Requisitos funcionais oferecem uma base natural para testes de funcionalidade e validação.
- Requisitos não funcionais podem exigir técnicas adicionais e critérios objetivos.
- Um bom caso de teste procura revelar defeitos antes que cheguem ao usuário.



O que é um caso de teste?

Conjunto de entradas, condições de execução e resultados esperados definido para atingir um objetivo específico de teste.

Na prática, o caso de teste direciona as ações do testador e explicita o que deve ser observado em cada passo.



Elementos de um caso de teste

Elemento	O que registra
Identificador	Código único do caso de teste
Objetivo	Intenção do teste
Entradas	Dados usados na execução
Saídas esperadas	Comportamento esperado
Ambiente	Hardware/software e versões necessárias
Procedimento	Como executar o teste
Dependências	Relação com outros casos de teste



Exemplo de caso de teste: login inválido

Item	Exemplo
Objetivo	Verificar tratamento de login inválido
Entrada	E-mail sem “@”: professor.gmail.com
Ação	Acionar a continuidade do login
Saída esperada	Indicar login inválido e permitir correção do campo
Ambiente	Smartphone com o aplicativo instalado



O que é um cenário (ou suíte) de teste?

Um cenário de teste agrupa casos de teste relacionados, geralmente associados ao mesmo componente ou funcionalidade.

- O agrupamento facilita organização, reexecução e testes de regressão.
- Também pode apoiar revalidação de mudanças e transferência de conhecimento.
- Na prática, o agrupamento também aparece como suíte ou ciclo de teste.



Caso de teste x cenário de teste

CASO DE TESTE

Uma sequência específica de entradas, condições, passos e resultados esperados.

CENÁRIO / SUÍTE



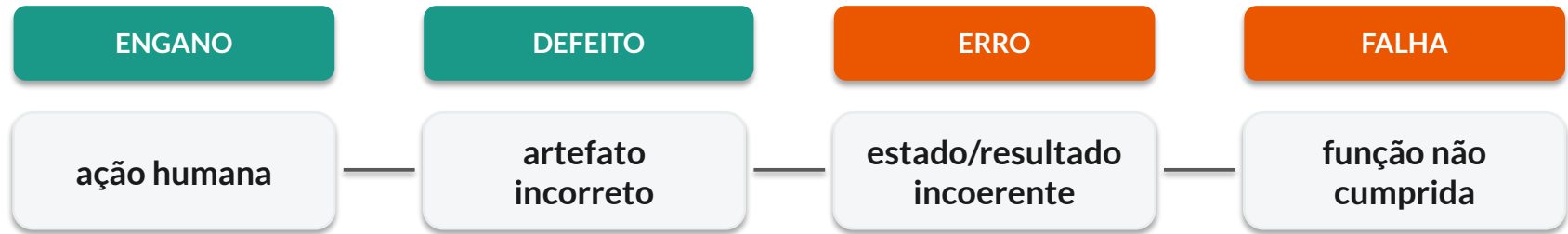
Um conjunto organizado de casos relacionados a uma mesma funcionalidade ou objetivo.



Engano, defeito, erro e falha

Termo	Definição resumida
Engano (mistake)	Ação humana que produz um resultado incorreto.
Defeito (fault)	Imperfeição/deficiência em um artefato que exige correção.
Erro (error)	Resultado incoerente produzido durante a execução.
Falha (failure)	Incapacidade do software de exercer a função para a qual foi desenvolvido.

A cadeia causal dos problemas



Um engano pode introduzir um defeito; quando o defeito é exercitado, pode produzir um erro e, conseqüentemente, uma falha observável.



Encontrou um defeito. E agora?

O defeito precisa ser reportado de forma que a equipe consiga reproduzir, analisar e corrigir o problema.

- O padrão de registro pode variar entre organizações e ferramentas.
- O princípio permanece: fornecer informação suficiente para localizar o comportamento observado.
- Passos, dados usados, versão e evidências visuais ajudam a reproduzir o problema.



Campos de um relatório de defeito

Campo	Finalidade
Identificador + descrição	Localizar e comunicar o defeito
Versão do produto	Contextualizar onde ocorreu
Passos + dados + evidências	Permitir reprodução
Data + reportado por	Apoiar rastreabilidade
Status	Acompanhar o tratamento
Corrigido por + encerramento	Registrar resolução
Severidade + prioridade	Apoiar avaliação e ordem de tratamento



Severidade x prioridade

SEVERIDADE

Expressa quão grave é o defeito para o sistema. Categorias: bloqueio de versão, crítico e pequeno.

PRIORIDADE

Define a prioridade de correção. Níveis alto, médio e baixo.



Teste não é depuração

TESTE

Executa casos de teste e produz resultados que podem revelar defeitos.

DEPURAÇÃO

Investiga o defeito encontrado, identifica sua causa e apoia a correção.

São atividades diferentes, mas a estratégia de depuração deve estar associada ao processo de teste.



Quando os testes terminam?

Pressman e Maxim destacam que não existe uma resposta definitiva e universal para a pergunta “quando parar de testar?”.

- Critérios de término precisam ser definidos de forma pragmática e mensurável.
- Modelos estatísticos podem apoiar estimativas sobre a confiabilidade alcançada.
- Na prática, o uso real do software continua exercitando o produto após a entrega.



Teste de regressão

Reexecutar testes já realizados para verificar se integrações ou mudanças introduziram efeitos indesejados.

- Mudanças podem criar novos fluxos de dados e novas combinações de entrada/saída.
- Partes anteriormente estáveis podem ser afetadas por uma modificação.
- O conjunto de regressão deve ser selecionado de forma criteriosa para equilibrar cobertura, tempo e custo.



Teste ao longo do ciclo de vida

A atividade de teste não deve ser tratada apenas como uma etapa final.

- Todos os modelos de processo discutidos por Martins incorporam atividades de teste.
- Em modelos iterativos e ágeis, o ciclo de planejar, implementar, integrar e testar se repete.
- Cada artefato produzido ao longo do desenvolvimento pode servir de base para atividades de teste.



Atividade 1 - escolhendo dados de teste

```
int exemplo_simples(int j) {  
    j = j - 1;  
    j = j / 30000;  
    return j;  
}
```

Em grupos: proponham um pequeno conjunto de valores de entrada e justifiquem por que esses valores foram escolhidos.

Pergunta-guia: quais valores têm maior chance de revelar um comportamento inadequado?



Atividade 2 - do teste ao relato de defeito

Considere o caso de teste de login inválido apresentado

Situação hipotética: o sistema aceita “professor.gmail.com” como login válido e não apresenta mensagem de validação.

- Quais informações mínimas devem constar no relato do defeito?
- Como você descreveria os passos para reprodução?
- Qual evidência seria útil anexar ao registro?



Referências

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de software: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021.

MARTINS, Marcos Danilo Chiodi. Testes de software. 1. ed. Rio de Janeiro: SESES, 2016.

IEEE. IEEE Std 829-2008: IEEE Standard for Software and System Test Documentation. New York: IEEE, 2008. IEEE Standards Association

ISO; IEC; IEEE. ISO/IEC/IEEE 24765:2010: Systems and software engineering — Vocabulary. Geneva: ISO, 2010. ISO

ISO; IEC. ISO/IEC 25010:2023: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model. Geneva: ISO, 2023.

FIRESMITH, Donald. Using V Models for Testing. Software Engineering Institute, Carnegie Mellon University, 11 nov. 2013. Disponível em: <https://www.sei.cmu.edu/blog/using-v-models-for-testing/>. Acesso em: 28 set. 2026.

MYERS, Glenford J. The Art of Software Testing. 2. ed. Hoboken, NJ: John Wiley & Sons, 2004.